

Aufgabe 1:

(a) $I \equiv S_1 = 20$

$$\Gamma \equiv S_1' = S_1 / 2 \quad P \equiv S_1 \bmod 2 \neq 0$$

$$\{S_1 \mapsto 20\} \rightarrow \{S_1 \mapsto 10\} \rightarrow \{S_1 \mapsto 5\}$$

$$\{S_1 \mapsto 2\} \rightarrow \{S_1 \mapsto 1\} \rightarrow \{S_1 \mapsto 0\}$$

oder

$$S_1 := 20$$

while true do

$$S_1 := S_1 / 2$$

end

$$\Gamma^{0,1} \equiv S_1' = S_1^0 / 2$$

$$\Gamma^{1,2} \equiv S_1'' = S_1' / 2$$

$$\Gamma^{2,3} \equiv S_1''' = S_1'' / 2$$

3-Sicherheit:

$$\Phi_{\text{3sich}} \equiv I \wedge \Gamma^{0,1} \wedge \Gamma^{1,2} \wedge \Gamma^{2,3} \Rightarrow P_0 \wedge P_1 \wedge P_2 \wedge P_3$$

$$\equiv S_1^0 = 20 \wedge (S_1' = S_1^0 / 2 \dots) \Rightarrow (S_1^0 \bmod 2 = 0 \wedge \dots)$$

3-Sicherheit-Test:

$$\Phi_{\text{3sich}} \equiv I \wedge \Gamma^{0,1} \wedge \Gamma^{1,2} \wedge \Gamma^{2,3} \wedge (\neg P_0 \vee \neg P_1 \vee \neg P_2 \vee \neg P_3)$$

$$\equiv S_1^0 = 20 \wedge S_1' = S_1^0 / 2 \wedge S_1'' = S_1' / 2 \wedge S_1''' = S_1'' / 2 \wedge (\neg S_1^0 \bmod 2 = 0 \vee \neg S_1' \bmod 2 = 0 \vee \neg S_1'' \bmod 2 = 0 \vee \neg S_1''' \bmod 2 = 0)$$

(c) Blatt 04.

Aufgabe 2:

Basis: $\mathbb{Z} \models I \models P \quad (I \Rightarrow P)$

$$\text{bzw. } S_1 = 1 \Rightarrow S_1 \bmod 2 \neq 0$$

$$S_1 = 1 \Rightarrow S_1 = 1 \wedge 1 \bmod 2 \neq 0 \Rightarrow S_1 \bmod 2 \neq 0$$

Step: $\mathbb{Z} \models P \wedge \Gamma \models P[S'/S]$

$$\text{bzw. } P \wedge \Gamma \Rightarrow P[S'/S]$$

$$P \wedge \Gamma \equiv S_1 \bmod 2 \neq 0 \wedge ((S_1 < 7000 \wedge S_1' = 3S_1) \vee (S_1 \geq 7000 \wedge S_1' = 1))$$

$$\equiv (S_1 \bmod 2 \neq 0 \wedge S_1 < 7000 \wedge S_1' = 3S_1) \vee (S_1 \bmod 2 \neq 0 \wedge S_1 \geq 7000 \wedge S_1' = 1) \Rightarrow S_1' \bmod 2 \neq 0 \equiv P[S'/S]$$

$$\Rightarrow S_1' \bmod 2 \neq 0$$

$$\Rightarrow S_1' \bmod 2 = 1 \Rightarrow S_1' \bmod 2 \neq 0$$

```
(declare-fun s_1 () int )
```

```
(declare-fun s_1' () int )
```

```
; I
(assert (= s_1 1))
```

```
; not P
(assert (not
  ; P
  (not (= ( mod s_1 2 ) 0))
))
```

```
; P and Gamma
(assert
  (not (= ( mod s_1 2 ) 0)
    (or
      (and ( < s_1 7000) (= s_1' (* 3 s_1)))
      (and ( >= s_1 7000) (= s_1' 1))
    )
  )
)
```

```
(assert (not
  (not(= (mod s_1pr 2))
))
```